

UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DCC 703 – COMPUTAÇÃO GRÁFICA
PROF DR LUCIANO F. SILVA

ANDREZA OLIVEIRA GONÇALVES

RASTERIZAÇÃO DE CIRCUNFERÊNCIAS

Relatório de implementação e resultados de algoritmos de rasterização de circunferências

Boa Vista/RR,
setembro de 2026

RASTERIZAÇÃO DE CIRCUNFERÊNCIAS **1**
1. Introdução e Objetivos **3**
2. Fundamentação Teórica **3**
Método da Equação Paramétrica 3
Método Incremental por Simetria 4
Método de Bresenham 4
3. Implementação **5**
4. Resultados **5**
5. Conclusão **7**

1. Introdução e Objetivos

A rasterização consiste na transição de gráficos vetoriais para estruturas matriciais, estabelecendo o desafio de traduzir equações matemáticas contínuas em uma grade discreta de pixels. Para compreender essa evolução tecnológica, este relatório analisa três abordagens clássicas dedicadas à rasterização de circunferências: o método fundamentado na equação paramétrica, a técnica incremental por simetria e o otimizado algoritmo de Bresenham.

2. Fundamentação Teórica

Para entender como funcionam os programas, é necessário entender os métodos:

Método da Equação Paramétrica

O método da equação paramétrica resolve o desenho de circunferências utilizando as funções trigonométricas de seno e cosseno baseadas em um parâmetro angular. Eis o funcionamento do cálculo:

- **Definição do passo angular:** O algoritmo define um incremento angular fixo $\Delta\theta$ inversamente proporcional ao raio, garantindo que os pontos gerados fiquem próximos o suficiente para evitar lacunas na curva.
- **Cálculo das coordenadas trigonométricas:** A cada iteração, o ângulo é incrementado, e as fórmulas $x = R.cos\theta$ e $y = R.sen\theta$ são resolvidas para determinar a posição cartesiana correspondente.
- **Arredondamento para a grade:** Os valores reais resultantes das funções trigonométricas são arredondados para o inteiro mais próximo, convertendo a posição contínua em um pixel discreto na tela.
- **Cálculo trigonométrico:** A expressão (math.pi * t) / 180 converte o ângulo atual t de graus para radianos, exigência das funções trigonométricas em Python. As equações paramétricas x = xc + r * math.cos(...) e y = yc + r * math.sin(...) calculam a posição geométrica exata do ponto subsequente da curva.

No método da equação paramétrica, a geração dos pontos que formam a circunferência ocorre de maneira sequencial e contínua, variando-se gradualmente o parâmetro angular. Embora essa abordagem ofereça uma formulação direta e intuitiva para definir geometricamente a curva no plano cartesiano, ela apresenta um custo computacional consideravelmente elevado. Essa ineficiência decorre principalmente da necessidade de calcular repetidamente funções trigonométricas de alto processamento, como seno e cosseno, a cada iteração do algoritmo. Além disso, como essas operações operam sobre valores de ponto flutuante, exige-se uma etapa adicional de arredondamento para mapear as coordenadas contínuas para a grade discreta de pixels da tela, o que pode introduzir imperfeições visuais e atrasos no desempenho geral.

Método Incremental por Simetria

O método incremental por simetria resolve o desenho eficiente de circunferências ao aproveitar as propriedades geométricas de espelhamento e calcular os pixels por meio de diferenças incrementais. Eis o funcionamento do código:

- **Simetria dos octantes:** O algoritmo calcula os pixels referentes a apenas um oitavo da circunferência (do ângulo de 0° a 45°); os demais sete octantes são preenchidos instantaneamente por reflexão simétrica das coordenadas em relação aos eixos e diagonais.
- **Cálculo incremental e diferenças finitas:** Em vez de reavaliar fórmulas complexas ou funções trigonométricas para cada ponto, o algoritmo atualiza a posição do pixel sucessivo utilizando somas e subtrações simples baseadas em uma variável de erro. Além disso, como cosθ e senθ são valores fixo, o algoritmo é otimizado.
- **Cálculo de t = 1 / r:** Ao definir o incremento do ângulo t como o inverso do raio, o código garante que o comprimento do arco percorrido a cada iteração seja de aproximadamente 1 pixel. Raio maior gera passo menor e raio menor gera passo maior.
- **Restrição ao primeiro octante:** Em vez de varrer 360 graus, o laço de repetição while y <= x avança apenas enquanto a coordenada y for menor ou igual à x (exatamente o trecho de 0° a 45°). Isso reduz drasticamente o número de iterações do laço principal. Os oito blocos sequenciais de append(round(...)) capturam a única coordenada (x, y) calculada nesta iteração e a replicam imediatamente para os outros sete octantes do plano. O código faz isso combinando permutações e inversões de sinais e, simultaneamente, soma os valores do centro (xc, yc) para transladar os pontos da origem para a posição final na tela.

Com o uso do método incremental por simetria, resolve-se o problema do alto custo de processamento das equações trigonométricas e analíticas e a descontinuidade da borda, pois o algoritmo calcula apenas uma fração da curva e expande o resultado de forma otimizada para toda a circunferência.

Método de Bresenham

O algoritmo de Bresenham para circunferências resolve o problema da eficiência e da precisão ao desenhar curvas circulares utilizando exclusivamente aritmética de números inteiros e explorando a simetria geométrica. Eis o funcionamento do algoritmo:

- **Exploração da simetria e ponto inicial:** O algoritmo calcula os pontos apenas para o primeiro octante (partindo de (0, R)), utilizando a reflexão simétrica de 8 octantes para preencher o restante da circunferência instantaneamente e definindo um parâmetro de decisão inicial com base diretamente no valor do raio.
- **Variável de decisão p = 1 - r:** O valor 1 - r é obtido matematicamente substituindo o ponto médio entre os dois primeiros pixels candidatos na equação implícita do círculo. A cada passo ao longo do eixo principal, um parâmetro de erro é avaliado para testar se o pixel candidato mais próximo encontra-se dentro ou fora da trajetória ideal da circunferência.

- **Atualização incremental:** Se p >= 0: O ponto médio teórico está fora (ou sobre) a curva exata. Isso significa que o pixel que melhor representa a circunferência é o da diagonal inferior. Portanto, o eixo secundário decresce (y = y - 1), a variável de decisão recebe uma grande compensação (p = p + 2*x - 2*y + 5) e o eixo principal avança (x = x + 1). No entanto, se p < 0: O ponto médio teórico está dentro da curva. O pixel mais fiel à trajetória está diretamente ao lado, na horizontal. O y se mantém (não sofre decremento), a variável de decisão sofre uma pequena compensação (p = p + 2*x + 3), e o eixo principal avança (x = x + 1).
- Conforme a varredura avança, o algoritmo atualiza o parâmetro de decisão e as coordenadas por meio de somas e subtrações simples, eliminando totalmente a necessidade de ponto flutuante, divisões ou funções trigonométricas.

Com o uso do método de Bresenham, resolve-se o problema do alto custo computacional e dos erros de arredondamento das abordagens analíticas e paramétricas, pois o algoritmo rasteriza circunferências com precisão exata e utiliza apenas números inteiros, garantindo ótima eficiência de hardware.

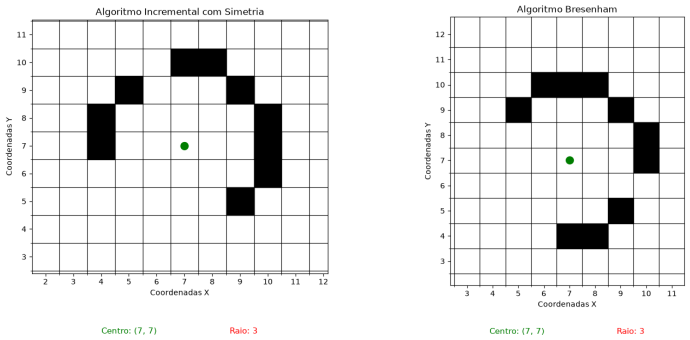
3. Implementação

- **Linguagem e ambiente:** O desenvolvimento foi estruturado na linguagem Python, utilizando a biblioteca *Matplotlib* para a plotagem e representação visual da grade discreta de pixels.
- **Desafios:** O principal desafio consistiu em transpor a lógica de um único octante para a geração completa da curva nos oito octantes a partir de um centro (xc, yc). Para alcançar essa abrangência sem o uso de ponto flutuante, no caso do Bresenham, a implementação emprega a variável de decisão do ponto médio (derivada da equação implícita da circunferência).
- **Controle de erro:** Inicializado a partir de um parâmetro baseado no raio (P = 3 - 2R), o algoritmo avalia o sinal da variável de decisão a cada passo no eixo principal para determinar se o próximo pixel avança na horizontal ou na diagonal. O uso dessa métrica, aliado à simetria dos 8 octantes, permite espelhar as coordenadas simultaneamente para todos os quadrantes, garantindo uma circunferência contínua, simétrica e computacionalmente otimizada.

4. Resultados

Inicialmente, em alguns dos testes do algoritmo incremental com simetria, observou-se a ausência de pixels na região inferior e na extremidade esquerda da grade. Após observar isso, a execução do algoritmo de Bresenham também apresentou um erro de renderização que omitiu integralmente a metade esquerda da figura geométrica.

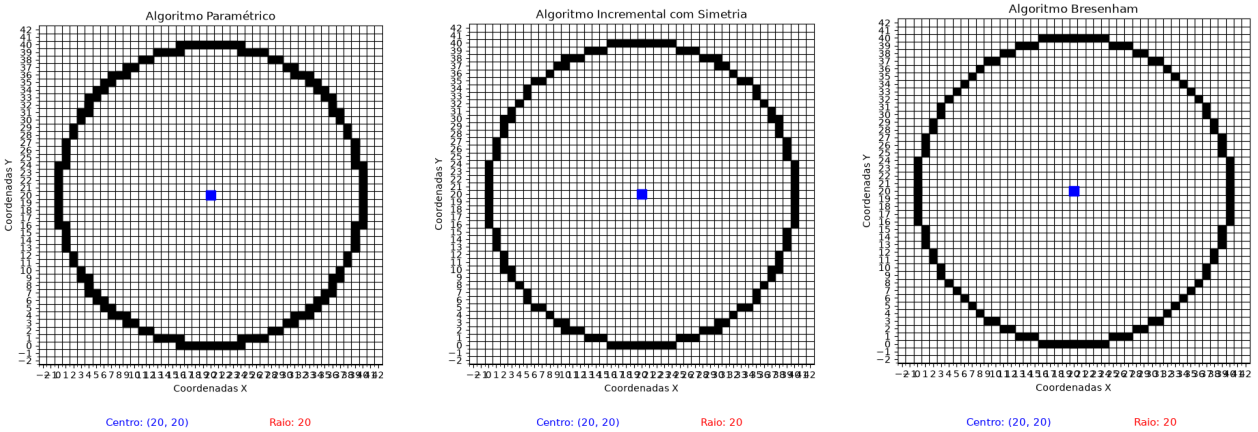
Figura 1 e 2 – Erros na rasterização das circunferências pelo algoritmo incremental e Bresenham



Fonte: Autoria própria, 2026

Essas anomalias indicaram que, embora o traçado do primeiro octante tenha sido calculado corretamente, ocorreram inconsistências na aplicação das inversões de sinal durante o espelhamento simultâneo para os demais quadrantes. No entanto, após observações e hipóteses, os algoritmos foram corrigidos com a lógica de octantes corretas, fazendo os devidos pares de x e y. Com isso, as circunferências ficaram completas em todos os três métodos. Vale ressaltar que o algoritmo paramétrico foi o único que não sofreu com este erro, pois não utiliza a lógica de simetria dos oito octantes, e por isso é consideravelmente mais lento também.

Figura 3, 4 e 5 – Rasterização da circunferência pelos três algoritmos



Fonte: Autoria própria, 2026

Os três algoritmos renderizaram com sucesso a circunferência completa na grade de pixels. O método paramétrico produziu um contorno fechado, contudo, a dependência de aproximações em ponto flutuante pode gerar aglomerações sutis de pixels ou espessura irregular em determinadas regiões da curva. O algoritmo incremental com simetria apresentou um traçado mais limpo e geometricamente uniforme, aproveitando-se do espelhamento simultâneo do primeiro octante para garantir a proporção da figura. Por fim, o algoritmo de Bresenham gerou um resultado visual de altíssima qualidade, com uma distribuição de pixels perfeitamente simétrica e sem redundâncias.

5. Conclusão

O estudo comparativo dos métodos de rasterização de circunferências evidenciou que as abordagens baseadas em equações paramétricas e no método incremental com simetria apresentam alto custo computacional devido à necessidade de resolver funções transcendentais (seno e cosseno) a cada passo, além de sofrerem com resíduos e erros de arredondamento decorrentes da aritmética de ponto flutuante.

O algoritmo de Bresenham soluciona essas deficiências ao eliminar completamente os números decimais, utilizando exclusivamente aritmética de inteiros e variáveis de decisão baseadas no ponto médio, o que confere ao desenho maior fidelidade à curva teórica. Desse modo, ao aliar essa precisão matemática à propriedade de simetria dos 8 octantes, o método garante máxima eficiência de processamento e integridade visual na grade de pixels, consolidando-se como a abordagem ideal para aplicações gráficas de alto desempenho.